《数据结构基础》(FDS) 专项练习题

Part 3: Trees, BST & Segment Trees

Based on ZJU Past Midterm Exams

说明：本卷包含讲义中【第三章：树与二叉树 (Trees & BST)】与【第六章：线段树 (Segment Trees)】的独立专项训练。至此，所有期中复习考点已全部覆盖。

Chapter 3: Trees & Binary Search Trees 专项练习

一、判断题 (True or False)

1-1. In a binary search tree (BST), the keys on the same level from left to right must be in sorted (non-decreasing) order. (T / F)

1-2. If the postorder and inorder traversal sequences of a binary tree are the same, then none of the nodes in the tree has a right child. (T / F)

1-3. If the pre-order traversal sequence of a binary tree is `A B C`, then `C A B` cannot be its in-order traversal sequence. (T / F)

1-4. To find 63 from a binary search tree, one possible searching sequence is {39, 101, 25, 80, 70, 59, 63}. (T / F)

1-5. A threaded binary tree with N nodes has exactly $N + 1$ threads (null links replaced by threads). (T / F)

1-6. Let n_0 be the number of leaf nodes and n_2 be the number of nodes of degree 2 in a non-empty binary tree. Then we always have $n_0 = n_2 + 1$. (T / F)

二、选择题 (Multiple Choice)

2-1. In a complete binary tree with 1102 nodes, there must be _____ leaf nodes.

A. 79

B. 551

C. 552

D. 1063

2-2. If A and B are both leaf nodes in a binary tree, then which of the following is TRUE?

A. There exists a binary tree with pre-order traversal sequence …A…B…and post-order traversal sequence …B…A….

B. There exists a binary tree with pre-order traversal sequence …A…B…and in-order traversal sequence …B…A….

C. There exists a binary tree with in-order traversal sequence …A…B…and post-order traversal sequence …B…A….

D. None of the above.

2-3. Insert {6, 9, 12, 3, 4, 8} one by one into an initially empty binary search tree. The post-order traversal sequence of the resulting tree is:

A. 3, 4, 9, 8, 12, 6

B. 3, 4, 6, 8, 12, 9

C. 4, 3, 8, 12, 9, 6

D. 4, 3, 6, 8, 12, 9

2-4. Given a binary tree with 2435 nodes. If it is a complete binary tree, how many leaf nodes does it have?

A. 1218

B. 1217

C. 812

D. Cannot be determined

三、程序填空题 (Fill in the blanks)

3-1. The following function `FindMin` is to find the minimum element in a Binary Search Tree (BST) iteratively. Please complete the code.

```
Position FindMin( SearchTree T ) {
    if ( T != NULL ) {
        while ( ________(1)________ ) {
            ________(2)________;
        }
    }
    return T;
}
```

四、函数题 (Programming)

4-1. No Less Than X in BST

You are supposed to output, in **decreasing order**, all the elements no less than ($\geq$) X in a binary search tree T.

```
void Print_NLT( Tree T, int X );
```

Note: `Tree` has `Element`, `Left`, and `Right`. Use `Output(int)` to print the element. Descending order is strictly required.

Chapter 6: Segment Trees 专项练习

一、判断题 (True or False)

1-1. The primary motivation for using a segment tree instead of a simple array is to allow $O(\log N)$ time complexity for both range queries and single element updates. (T / F)

1-2. Building a segment tree from an array of N elements takes $O(N \log N)$ time. (T / F)

1-3. A segment tree representing an array of size N is always a perfectly balanced full binary tree. (T / F)

1-4. To store a segment tree for an array of N elements in a 1D array layout (like a heap), an array of size $4N$ is always sufficient. (T / F)

二、选择题 (Multiple Choice)

2-1. What is the time complexity to query the sum of an arbitrary range $[L, R]$ in a Segment Tree built on an array of size N?

A. $O(1)$

B. $O(\log N)$

C. $O(N)$

D. $O(N \log N)$

2-2. When updating the value of a single element at index i in an array of size N, how many nodes in the Segment Tree will be updated?

A. exactly 1

B. exactly N

C. $\approx \log_2 N$

D. $\approx N/2$

三、程序填空题 (Fill in the blanks)

3-1. The following function `Query` finds the sum of elements in range `[L, R]` using a Segment Tree. `node` is the current tree index, covering range `[start, end]`. Complete the code.

```
int Query(int tree[], int node, int start, int end, int L, int R) {
    if (R < start || L > end) {
        return 0; // Completely outside
    }
    if (________(1)________) {
        return tree[node]; // Completely inside
    }
    int mid = (start + end) / 2;
    int left_sum = Query(tree, 2 * node, start, mid, L, R);
    int right_sum = ________(2)________;

    return left_sum + right_sum;
}
```

参考答案与解析

Chapter 3: Trees & Binary Search Trees

一、判断题

1. **T**. In a BST, any node in the left subtree is smaller, and in the right is larger. If we look at nodes on the same level from left to right, their values must be non-decreasing.

2. **T**. Postorder is Left-Right-Root. Inorder is Left-Root-Right. If they are exactly the same, there can be no Right child anywhere, otherwise the Root would appear before Right in Inorder, but after Right in Postorder.

3. **T**. Preorder is A-B-C. Root is A. If Inorder is C-A-B, then C is left child, B is right child. The pre-order of this tree would be A-C-B, which contradicts A-B-C.

4. **F**. Searching for 63. Sequence: 39 (go right), 101 (go left), 25 (ERROR!). Since 25 is in the right subtree of 39, it must be > 39. Finding 25 here violates the BST property.

5. **T**. A standard binary tree with N nodes has $2N$ total pointers. $N-1$ of them are used for actual branches. Thus, there are $2N-(N-1)=N+1$ NULL pointers, which become threads.

6. **T**. This is a fundamental property of all binary trees: $n_0 = n_2 + 1$.

二、选择题

1. **B**. For a complete binary tree with N nodes, the number of leaf nodes $n_0 = \lceil N/2 \rceil$. So $\lceil 1102/2 \rceil = 551$.

2. **A**. Option A is true because A can be in the left subtree and B in the right subtree. Pre-order visits left then right (…A…B…). Post-order visits left then right then root (…A…B…root). Wait! If post-order is …B…A…, it means A is visited AFTER B. For two leaf nodes, post-order Left-Right-Root means Left leaf is printed before Right leaf. So if A is left and B is right, post-order is A…B. For post-order to be B…A, B must be left and A must be right. But in Pre-order, Left is printed before Right, so Pre-order would be B…A. Thus A is FALSE. *Correct answer is B*. In Option B, A is visited before B in Pre-order, but B is visited before A in In-order. This happens when A is an ancestor of B, but they are LEAVES! If they are both leaves, Left leaf is always before Right leaf in Pre-order, In-order, AND Post-order. Thus, none of A, B, C is possible! **Answer is D**.

3. **C**. Insertions: 6(root) → 9(right) → 12(right of 9) → 3(left of 6) → 4(right of 3) → 8(left of 9). Post-order: Left-Right-Root → (null, 4, 3), (8, 12, 9), 6 → 4, 3, 8, 12, 9, 6.

4. **A**. $N = 2435$. Leaf nodes $= \lceil 2435/2 \rceil = 1218$.

三、程序填空题

1. `T->Left != NULL` (在 BST 中找最小值，需要一直往左子树走，直到没有左孩子)
2. `T = T->Left` (不断更新当前节点为其左孩子)

四、函数题

```
void Print_NLT( Tree T, int X ) {
    if ( T == NULL ) return;

    /* To print in DECREASING order, we must visit RIGHT subtree first */
    /* We only need to visit right subtree if it exists */
    Print_NLT( T->Right, X );

    /* Print the current node if it satisfies >= X */
    if ( T->Element >= X ) {
        Output( T->Element );

        /* If current node >= X, the left subtree MIGHT contain
           elements >= X, so we must explore it */
        Print_NLT( T->Left, X );
    }
    /* If T->Element < X, then everything in its left subtree
       is also < X, so we DO NOT visit the left subtree to save time. */
}
```

Chapter 6: Segment Trees

一、判断题

1. **T**. A normal array takes $O(1)$ for updates but $O(N)$ for sum queries. A segment tree balances this to $O(\log N)$ for both.
2. **F**. Building a segment tree takes $O(N)$ time because each node is visited once and computed in $O(1)$ time from its two children.
3. **F**. A segment tree is a balanced binary tree, but not necessarily a "perfectly full" binary tree (unless N is a power of 2).
4. **T**. It is mathematically proven that a 1D array of size $4N$ is always sufficient to store a segment tree for an array of size N.

二、选择题

1. **B**. Range sum queries recursively split at most into 4 nodes per level, yielding $O(\log N)$.
2. **C**. A point update only changes the leaf node corresponding to that index and its ancestors up to the root, which is exactly the height of the tree, $\approx \log_2 N$.

三、程序填空题

1. `L <= start && end <= R` (如果当前节点代表的区间 `[start, end]` 完全被目标查询区间 `[L, R]` 包围，则直接返回该节点的值，这是线段树能达到 $O(\log N)$ 的核心剪枝)
2. `Query(tree, 2 * node + 1, mid + 1, end, L, R)` (递归查询右半区间，右孩子节点索引为 `2*node+1`，范围为 `mid+1` 到 `end`)